

The Anatomy of a Password: Predicting Crack Time

Gurman Basran

April 2026

DASC 4850, Foundations of Data Science and AI in Python

University of Lethbridge

Introduction

I run a self hosted password manager called Vaultwarden as part of my homelab and ive always been curious about whether the standard workplace password advice (8 characters minimum, mix upper and lower case, add a digit, add a special character) actually holds up against real attackers. Every IT department says some version of those rules but ive never seen anyone show me the data. So this project is me sitting down and testing it with real breach data and a regression model.

The setup: every website that stores user passwords runs them through a one way hash function before storing them, so a database leak gives attackers hashes rather than plaintext. Modern GPUs can hash around 10 billion candidate passwords per second against the leaked dump and check for matches [1]. Stack a few cards together and youre at hundreds of billions per second. Attackers also dont guess randomly; they start with breach lists and dictionary attacks with standard modifications (capitalize, swap a for @, append digits) and only fall back to full brute force when nothing else works. The crack time for any given password is whichever of those attacks finds it first.

I built a linear regression model that predicts $\log_{10}$ of the crack time in seconds for a password based on measurable properties of the string itself. The dataset combines about 10,000 sampled real passwords from breach data (RockYou 2009 [2], SecLists [3], the Probable Wordlists collection covering LinkedIn, Adobe, and Dropbox [4], and the Pwdb 2021 dump from the COMB compilation [5, 6]) with 2,000 generated strong passwords representing what a password manager would output. That gives me both ends of the spectrum so the model can learn the full range of crack times.

Data and Feature Engineering

After deduplication and minimum length filtering I had around 12,000 passwords to work with. For each password I extracted 10 numeric and categorical features. The numeric ones cover length, Shannon entropy [7] ($H = -\sum p_i \log_2 p_i$), character set size, ratio of unique to total characters, and binary flags for whether the password contains a dictionary word, a keyboard pattern (qwerty, 1qaz2wsx), a sequential run (abc, 123), or appears in a known breach list. The categorical ones are character set category (digits only through all four character types) and length bucket (very short through very long).

The target variable is $\log_{10}$ of estimated crack seconds. The estimation function tries every attack type the password is vulnerable to and uses whichever finishes fastest: a breach list lookup if the password is in known leaked dumps, a dictionary attack if it contains a common word (around 100k words times standard modifications times brute force on any extra characters), a pattern attack if it contains a keyboard walk, and full brute force otherwise (charset size raised to the password length). Everything divided by 10 billion guesses per second, a benchmark consistent with hashcat performance numbers on consumer GPUs against fast hashes like MD5 or SHA1 [1]. The estimation methodology is conceptually similar to the way zxcvbn [8] computes guessability but uses a simpler additive attack model. Raw crack times span 32 orders of magnitude from microseconds to longer than the universe has existed, which is why the log transformation is necessary; it turns an unmodelable range into something a linear regression can actually fit.

Exploratory Data Analysis

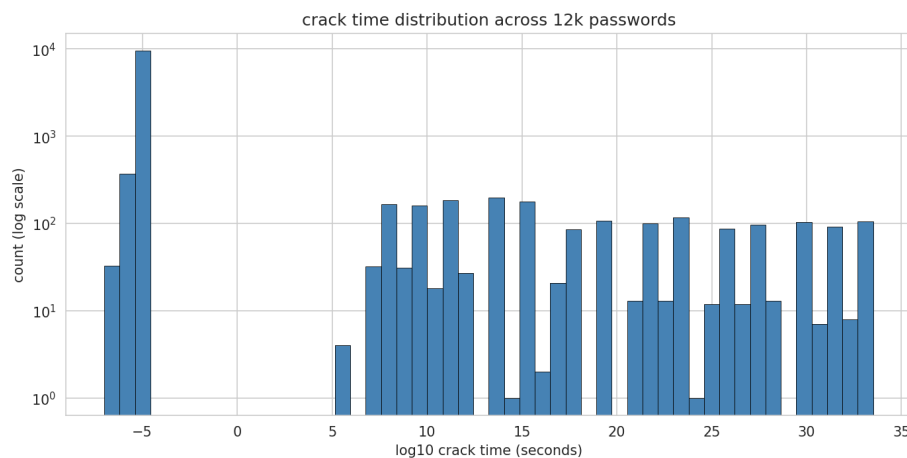


Figure 1. Crack time distribution across all 12k passwords on a log y axis. The huge spike on the left is real breach passwords getting cracked in under a second. The flat strip on the right is generated strong passwords. The gap in the middle is the part that shows the actual problem: nobody picks passwords that land between weak and strong, you either reuse password123 or let a manager generate something random.

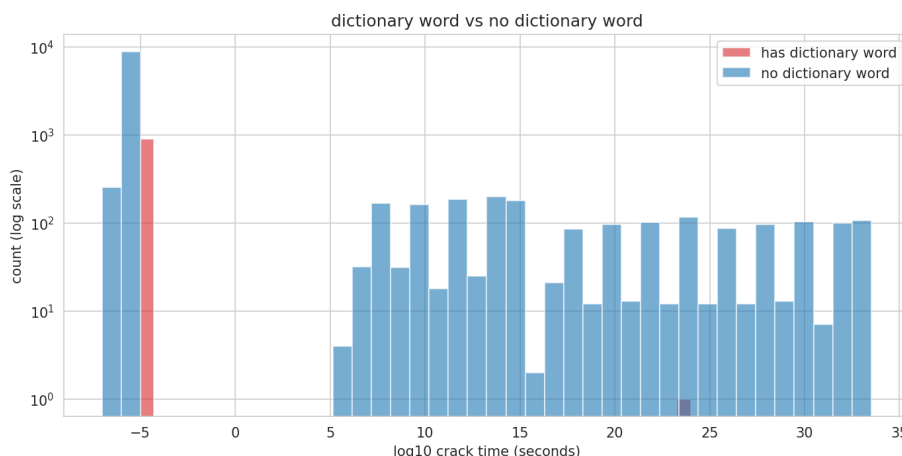


Figure 2. Same data split by whether the password contains a dictionary word. Red is jammed all the way to the left, dictionary attacks find these in microseconds regardless of how many ! and 123 get tacked on. Blue spreads across the entire range because random looking strings have to be brute forced.

Statistical Test

The plots make it look obvious that dictionary words are catastrophic, but I formally tested it with a Welch two-sample t-test on log crack time between the two groups. The result: p-value essentially zero (well below the 0.05 threshold), so the null hypothesis of no difference is rejected. Cohens d came out to around 0.6 which sits between the conventional thresholds for medium (0.5) and large (0.8) effect sizes. The reason d isn't larger despite the visual gap is that the no-dictionary group has a huge variance (it contains both weak breach passwords and strong generated ones), which inflates the pooled standard deviation. The mean difference is what tells the practical story: passwords without dictionary words are around $10^{4.15} \approx 14,000\times$ harder to crack on average. The 95% confidence interval doesn't include zero, confirming the result a third way.

Predictive Modeling

I used Ridge regression [9] rather than plain OLS because the heatmap showed correlated features (length and entropy in particular, plus the categorical buckets overlapping with the numeric features), and OLS coefficients become unstable when $X^T X$ is ill conditioned. Ridge adds a small regularization penalty to the diagonal which keeps coefficients stable, basically Tikhonov regularization. The regularization strength α was tuned via 5 fold cross validation through GridSearchCV across 13 candidate values from 0.001 to 1000. Numeric features were standardized and the two categorical features one hot encoded, all wrapped in a scikit-learn [10] Pipeline so the preprocessor steps cant leak data between train and val.

The data was split 60/20/20 into train, validation, and test sets. The model picks the best α based on validation set R-squared, then the test set is used exactly once at the end for an unbiased estimate of generalization performance.

Metric	Validation	Test
RMSE	1.271	1.032
MAE	0.538	0.531
R-squared	0.981	0.988

Table 1. Final model performance. RMSE is in $\log_{10}$ units so a value of 1.0 means predictions are typically off by about one order of magnitude (predicting 10 seconds when the true answer is 100 seconds). R^2 of 0.988 means the model explains about 99% of the variance in crack time across 32 orders of magnitude, which is honestly more than I expected from a linear model.

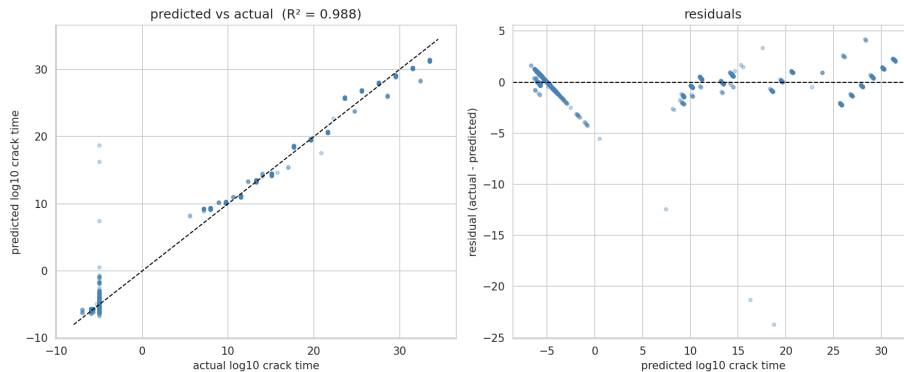


Figure 3. Predicted versus actual log crack times on the test set, plus residuals on the right. The dots hug the diagonal closely across the

entire range. Residuals fan slightly wider in the middle because the very weak and very strong passwords are easier to predict (one is always near the floor, the other always near the ceiling) while the middle region is where the model has to actually work.

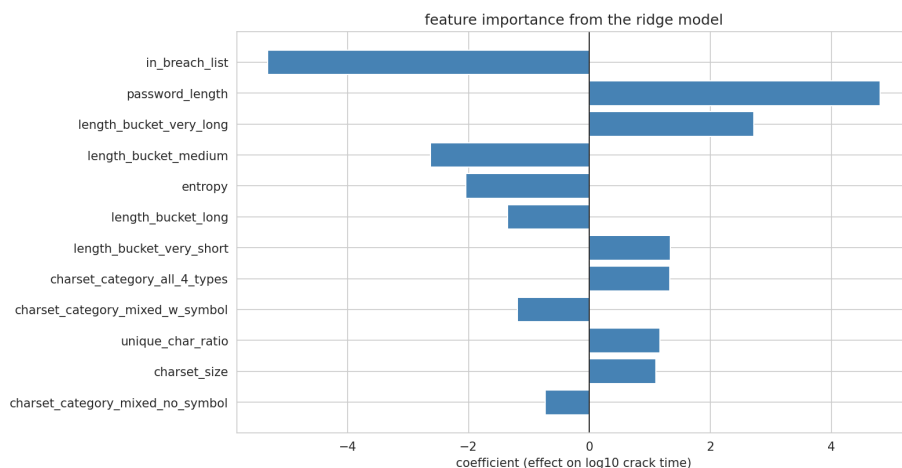


Figure 4. Top 12 feature coefficients from the Ridge model. `password_length` dominates as the largest positive predictor and `in_breach_list` dominates as the largest negative predictor. Some categorical buckets appear with both signs because the model splits effect across correlated columns.

Interpretation

The biggest finding by a wide margin is that password length absolutely dominates everything else in the model. Every additional character multiplies the brute force search space by the size of the character set, which is exponential growth, and exponential growth always wins eventually. A 20 character lowercase passphrase like `correcthorsebatterystaple` has $26^{20} \approx 2 \times 10^{28}$ possible combinations. An 8 character mix of all four character types like `K$3p9!aB` has $95^8 \approx 7 \times 10^{15}$ combinations. The lowercase one is about a trillion times stronger despite using a smaller alphabet, just because of length. The data says length matters way more than the complexity rules every workplace policy emphasizes.

The second finding is that dictionary words are the single biggest vulnerability. `has_common_word` and `in_breach_list` both light up huge in the model and the t-test confirmed dictionary words specifically. So `Monkey123!` is essentially as crackable as `monkey`, the capitalization and digits and exclamation point all feel like security but they're standard modifications that crackers test automatically. Those tricks

worked maybe in the early 2000s when wordlists were smaller but its been at least 15 years since they actually slowed anyone down.

Character types do help but with diminishing returns. Going from digits only to letters only is a huge jump (alphabet goes from 10 to 26 characters) but adding a digit on top of letters, then a symbol, then both, each one helps less than the last. Charset is the base of the exponent and length is the exponent itself, so length wins.

The practical advice that follows from all this is straightforward. Use a password manager (Vaultwarden, Bitwarden, 1Password, KeePass) and let it generate 16+ character random strings for every account. For the master password you actually have to memorize, use a 4 or 5 word random passphrase from a wordlist (the EFF diceware list [11] is the standard recommendation) because the length gives crack resistance and random word selection prevents whole phrase attacks. Stop using "complex" 8 character passwords with symbols, theyre much weaker than 16 lowercase letters even though they look smarter.

A few caveats on the absolute numbers. The crack time estimates assume the website used MD5 or similar fast hash function at 10 billion guesses per second; modern systems should use slower algorithms like bcrypt [12] or Argon2 [13] which slow that down to thousands per second instead of billions. That shifts all my numbers but doesnt change the relative importance of features. Also the dictionary I used to detect "dictionary words" only has about 130 common english words, while real cracking tools use wordlists with millions of entries, so my model is actually optimistic about how strong dictionary based passwords are.

Conclusion and Next Steps

The model achieves R^2 of 0.988 on a held out test set across 32 orders of magnitude in crack time, with length as the dominant positive predictor and dictionary word presence as the dominant negative one. These findings line up with the NIST 2017 password guideline update [14] which moved away from complexity rules toward longer simpler passwords, but seeing it quantitatively in my own breach data made it click in a way reading the NIST update never did. The next thing I want to do is integrate this analysis into my Vaultwarden setup as a real time password strength checker; the features I engineered here are the same ones production tools like zxcvbn [8] use, and a simple rule based system using just the top three features (length, dictionary check, charset complexity) would already catch the majority of weak passwords being entered.

AI Use Appendix

For this project I picked the topic myself (running Vaultwarden on my homelab made me curious about what actually makes a password hard to crack instead of just trusting workplace IT advice), sourced all the breach datasets, and decided which 10 features to extract per password based on what I knew from running my homelab and reading hashcat output. A lot of the modeling decisions came from MATH 3850 (numerical optimization and machine learning) which I'm taking concurrently. Choosing Ridge over plain OLS came from knowing that correlated features make plain regression unstable and a small regularization penalty fixes that, which we covered in class. Same with understanding why you standardize features before fitting, why you use cross validation to pick the regularization strength, and why the train/val/test split needs three sets when you're tuning hyperparameters. The 60/20/20 split and the two sample t-test both came from class too.

Where AI actually helped was on implementation rather than concepts. I sketched out the priority logic for the crack time formula myself (which attack happens first, why a breach lookup beats brute force) but had AI formalize the actual math for each attack type. The Welch's t-test setup and Cohen's d formula were AI assisted since I knew I needed effect size but couldn't remember the pooled standard deviation formula off the top of my head. The scikit learn pipeline with the preprocessor and grid search was more API plumbing than I had memorized, so AI helped me get the parameter signatures right and made sure data wouldn't leak between train and val. AI also debugged a few feature engineering edge cases. Every analysis decision and every interpretation is mine though, and I made sure I could defend any line of code AI helped me write.

References

- [1] Hashcat advanced password recovery. Official documentation and benchmarks. <https://hashcat.net/wiki/>
- [2] RockYou wordlist (rockyou-75.txt subset of the 2009 32M account breach), curated by SecLists. <https://github.com/danielmiessler/SecLists/blob/master/Passwords/Leaked-Databases/rockyou-75.txt>
- [3] Miessler, D., et al. SecLists: a security tester's companion. <https://github.com/danielmiessler/SecLists>
- [4] berzerk0, Probable-Wordlists. Aggregated real-world password lists from LinkedIn 2012, Adobe 2013, Dropbox 2016, and others. <https://github.com/berzerk0/Probable-Wordlists>

- [5] ignis-sec, Pwdb-Public (2021). Password analysis derived from 2 billion+ leaked credentials. <https://github.com/ignis-sec/Pwdb-Public>
- [6] Cybernews, "Mother of All Breaches: a 26 billion record compilation discovered" (Jan 2024). <https://cybernews.com/security/billions-passwords-credentials-leaked-mother-of-all-breaches/>
- [7] Shannon, C. E., "A Mathematical Theory of Communication," Bell System Technical Journal, 27(3), 379-423 (1948). <https://people.math.harvard.edu/~ctm/home/text/others/shannon/entropy/entropy.pdf>
- [8] Wheeler, D. L., "zxcvbn: Low-Budget Password Strength Estimation," 25th USENIX Security Symposium (2016). <https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/wheeler>
- [9] Hoerl, A. E., Kennard, R. W., "Ridge Regression: Biased Estimation for Nonorthogonal Problems," Technometrics, 12(1), 55-67 (1970). <https://www.tandfonline.com/doi/abs/10.1080/00401706.1970.10488634>
- [10] Pedregosa, F., et al. "Scikit-learn: Machine Learning in Python." Journal of Machine Learning Research, 12, 2825-2830 (2011). <https://scikit-learn.org/>
- [11] Electronic Frontier Foundation, "EFF's Long Wordlist for Random Passphrases." <https://www.eff.org/dice>
- [12] Provos, N., Mazières, D., "A Future-Adaptable Password Scheme," USENIX Annual Technical Conference (1999). bcrypt specification. <https://www.usenix.org/legacy/event/usenix99/provos/provos.pdf>
- [13] Biryukov, A., Dinu, D., Khovratovich, D., "Argon2: the memory-hard function for password hashing and other applications" (2017). <https://www.password-hashing.net/argon2-specs.pdf>
- [14] National Institute of Standards and Technology, Special Publication 800-63B, "Digital Identity Guidelines: Authentication and Lifecycle Management" (2017). <https://pages.nist.gov/800-63-3/sp800-63b.html>
- [15] Vaultwarden, self-hosted Bitwarden-compatible password manager (homelab deployment context). <https://github.com/dani-garcia/vaultwarden>